Actividades CodeWars dificultad 8 y 7



Alex Sanz
2ºDAM

ÍNDICE

Actividades dificultad 8 **3**

1.Switch it Up! 3

2.Add new item 3

3.MakeUpperCase 4

4.Generate range of integers 4

5.Evil or Odious 5

6.Even or Odd 6

7.Square(n) Sum 6

8.Multiply 7

9.String repeat 7

10.Convert a Boolean to a String 8

Actividades dificultad 7 **8**

1.Leap Years 8

2.You’re a Square! 9

3.Highest and Lowest 9

4.Descending Order 10

5.Multiply Adjacent Digits 10

Actividades dificultad 8

1.Switch it Up!

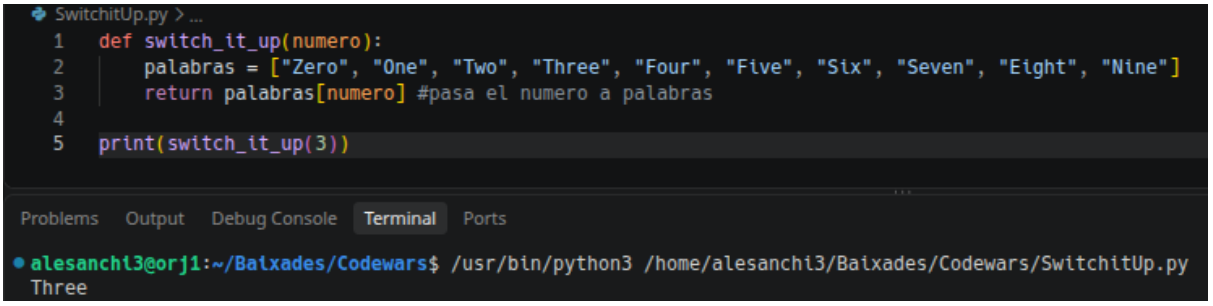
Proporciona un número entre `0-9` con palabras.

```python
def switch_it_up(numero):
    """Convierte un número entero (del 0 al 9) en su representación escrita en inglés.

    PARAMETERS
    ---
    numero (int): El número entero que se desea pasar a texto.

    RETURNS
    ---
    str: La palabra en inglés correspondiente al índice del número proporcionado.
    """
    palabras = ["Zero", "One", "Two", "Three", "Four", "Five", "Six", "Seven", "Eight", "Nine"]
    return palabras[numero]


print(switch_it_up(3))  # Salida : Three (Numero a texto)
```



Se crea un diccionario o array y luego devolvemos ese diccionario apuntando a el número puesto.

2.Add new item

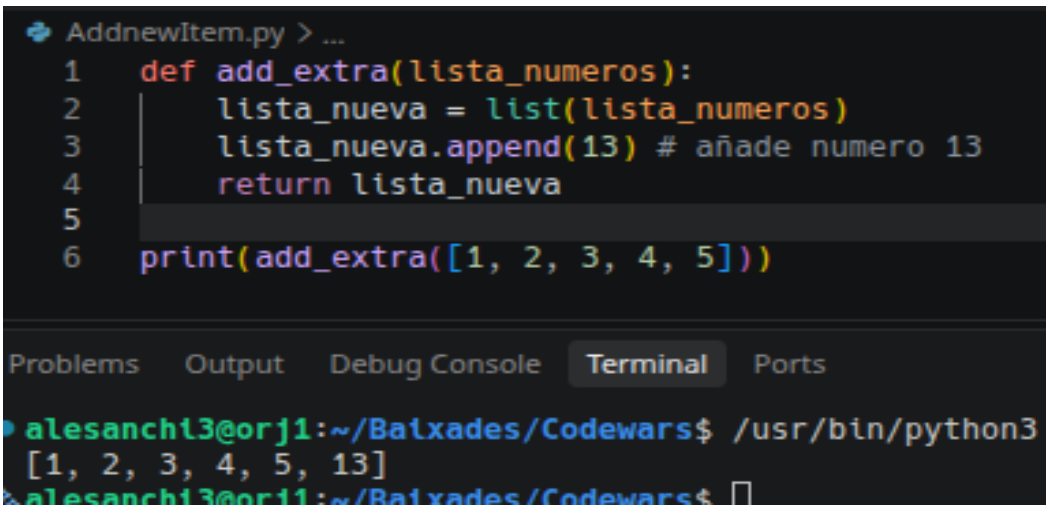
Añade un nuevo elemento a la lista y devuelve la lista. El nuevo elemento puede ser cualquier objeto, y no importa. (digamos que agrega un valor entero, como 13).

```python
def add_extra(lista_numeros):
    """Crea una nueva lista a partir de la proporcionada y le añade el número 13 al final.

    PARAMETERS
    ---
    lista_numeros (list/iterable): La lista original de números a la que se le añadirá un elemento.

    RETURNS
    ---
    list: Devuelve `lista_nueva`, una copia de la lista original con el número 13 añadido al final.
    """
    lista_nueva = list(lista_numeros)  # copia la lista list() para convertir en lista en caso de que no sea lista
    lista_nueva.append(13)  # añade numero 13
    return lista_nueva  # devuelve la lista con el numero añadido


print(add_extra([1, 2, 3, 4, 5]))  # salida : lista junto el nuevo numero
```



Se hace un clon de la lista y luego un .append() para añadir ese número a la lista.

3.MakeUpperCase

Escriba una función que convierta la cadena de entrada en mayúsculas.

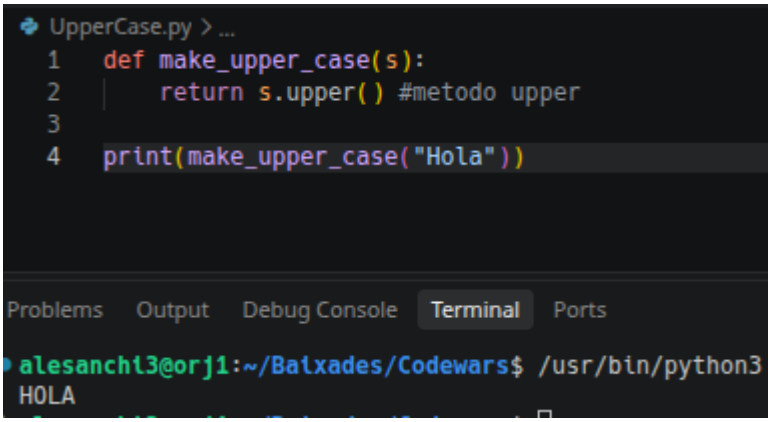
```python
def make_upper_case(s):
    """Convierte todos los caracteres de una cadena de texto a mayúsculas.

    PARAMETERS
    ---
    s (str): La cadena de texto que se desea transformar.

    RETURNS
    ---
    str: Devuelve el resultado de aplicar el método upper() sobre la cadena, es decir, el texto completamente en mayúsculas.
    """
    return s.upper()  # método upper


print(make_upper_case("Hola"))  # Salida: HOLA (el texto en mayús)
```

Hacer un .upper() para hacerlo en mayúsculas.



4.Generate range of integers

Implementar la función generateRange que toma tres argumentos (start, stop, step)Y devuelve el rango de enteros de starta stop(inclusive) en incrementos de step.

```python
def generate_range(start, stop, step):
    """Genera una lista de números desde un valor inicial hasta un valor final con un incremento determinado.

    PARAMETERS
    ---
    start (int): El número inicial del rango.
    stop (int): El número final del rango (incluido).
    step (int): El incremento o salto entre cada número de la secuencia.

    RETURNS
    ---
    list: Devuelve la lista con la secuencia de números generada mediante `range(start, stop + 1, step)`.
    """
    return list(range(start, stop + 1, step))  # secuencia de números


print(generate_range(1, 10, 1))  # salida: lista del 1 al 10 de 1 en 1
```



Se pone en list() para hacerlo en lista, range() crea una secuencia de números con los parámetros de inicio, fin y step para el intervalo.

5.Evil or Odious

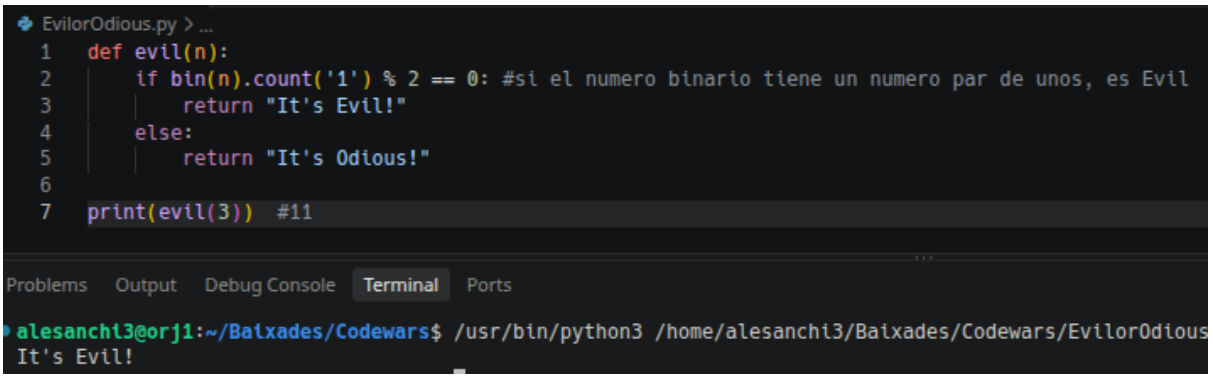
El número entero es Evil si tiene un número par de 1 en su representación binaria.
El número entero es Odious si tiene un número impar de 1 en su representación binaria.

```python
def evil(n):
    """Determina si un número es 'Evil' (tiene un número par de unos en su representación binaria) o 'Odious' (número impar de unos).

    PARAMETERS
    ---
    n (int): El número entero que se desea evaluar.

    RETURNS
    ---
    str: Devuelve la cadena "It's Evil!" si la cantidad de unos en binario es par, o "It's Odious!" si es impar.
    """
    if bin(n).count('1') % 2 == 0:  # si el numero binario tiene un numero par de unos, es Evil
        return "It's Evil!"
    else:
        return "It's Odious!"


print(evil(3))  # 11
```



Usar bin() para que los transforme en binario, count() cuenta el número de 1 y % 2 == 0 comprueba si es par, una vez ya hecho eso falta printear el resultado .

6.Even or Odd

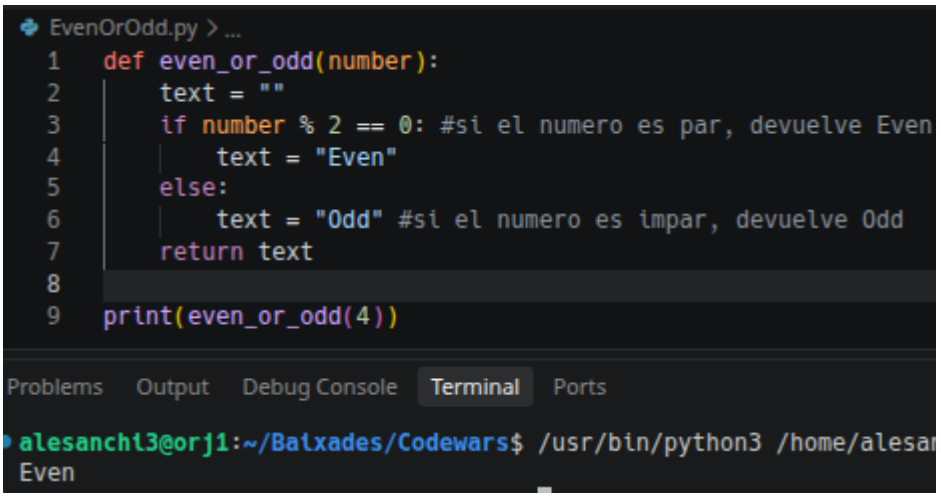
Crear una función que tome un entero como argumento y regrese "Even"Para números pares o "Odd"Para números impares.

```python
def even_or_odd(number):
    """Determina si un número entero es par o impar.

    PARAMETERS
    ---
    number (int): El número entero que se va a evaluar.

    RETURNS
    ---
    str: Devuelve la variable `text` con el valor "Even" si el número es par, o "Odd" si es impar.
    """
    text = ""
    if number % 2 == 0:  # si el numero es par, devuelve Even
        text = "Even"
    else:
        text = "Odd"  # si el numero es impar, devuelve Odd
    return text


print(even_or_odd(4))
```



Este es similar al anterior es solo usar n%2==0 para ver si es par y luego printear el resultado.

7.Square(n) Sum

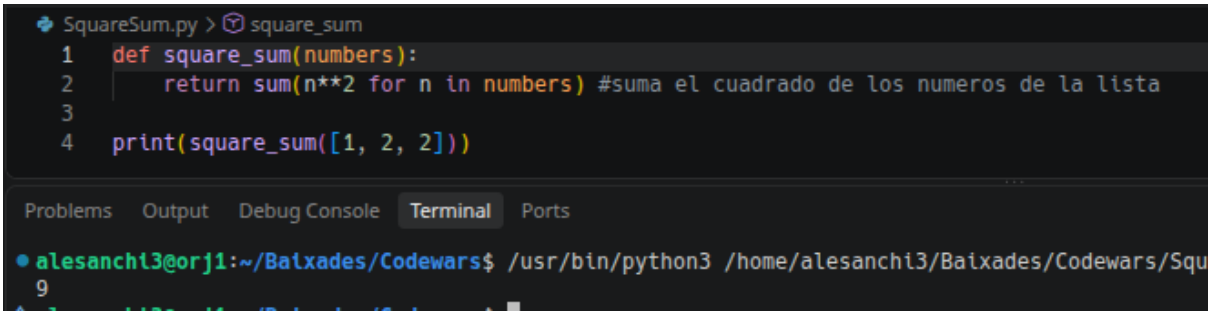
Suma cuadrada de modo que cuadra cada número que pasa a ella y luego suma los resultados juntos.

```python
def square_sum(numbers):
    """Calcula la suma de los cuadrados de todos los números contenidos en una lista o iterable.

    PARAMETERS
    ---
    numbers (list/iterable): Una secuencia o lista de números que se van a elevar al cuadrado y sumar.

    RETURNS
    ---
    int/float: Devuelve el resultado de sumar los cuadrados de cada elemento, obtenido mediante la función `sum()`.
    """
    return sum(n ** 2 for n in numbers)  # suma el cuadrado de los numeros de la lista


print(square_sum([1, 2, 2]))
```



Se hace sum() para sumar el resultado de el for, n2 es para la raíz cuadrada y por último hacer el for para la cantidad de números.**

8.Multiply

Este código no se ejecuta correctamente. Trate de averiguar por qué.

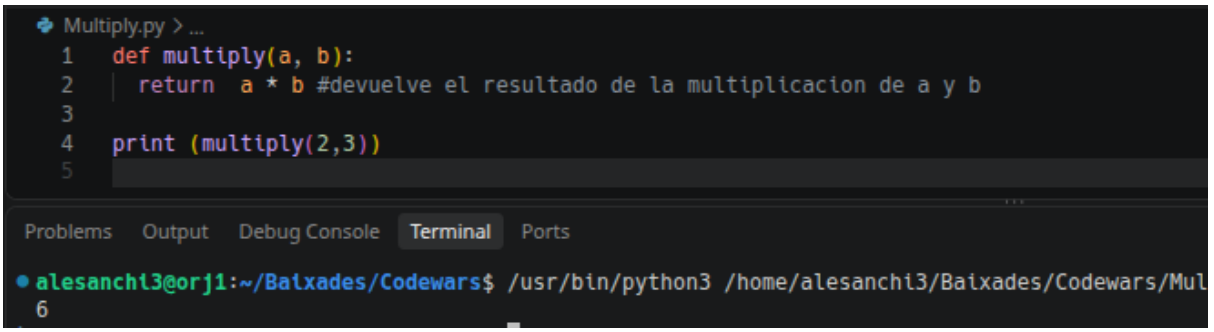
```
def multiply(a, b):
    """Multiplica dos números y devuelve el resultado de la operación.

    PARAMETERS
    ---
    a (int): El primer número (factor) a multiplicar.
    b (int): El segundo número (factor) a multiplicar.

    RETURNS
    ---
    int: Devuelve el resultado de la multiplicación de `a` y `b`.
    """
    return a * b  # devuelve el resultado de la multiplicacion de a y b


print(multiply(2, 3))
```

```
def multiply(a, b):
    a * b
```



Faltaba el return.

9.String repeat

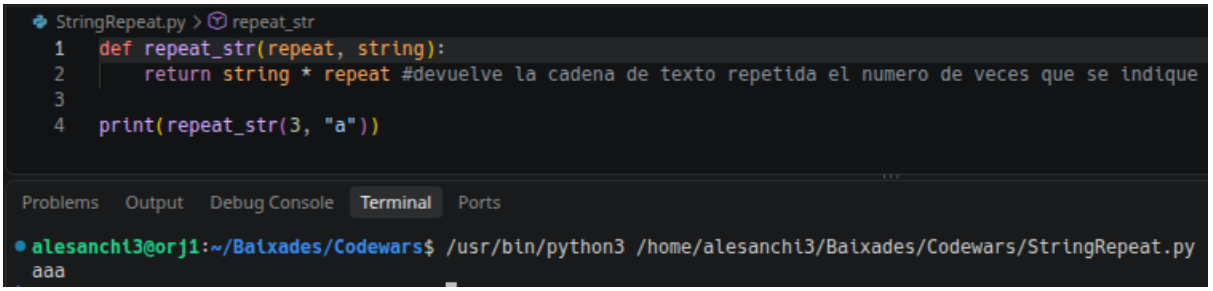
Un número entero no negativo Y un string como parámetros, devuelve una cadena repetidamente con la cantidad del número.

```python
def repeat_str(repeat, string):
    """Repite una cadena de texto un número determinado de veces.

    PARAMETERS
    ---
    repeat (int): El número de veces que se desea repetir la cadena.
    string (str): La cadena de texto que se va a repetir.

    RETURNS
    ---
    str: Devuelve el resultado de multiplicar la cadena por el número
de repeticiones (`string * repeat`).
    """
    return string * repeat  # devuelve la cadena de texto repetida
el numero de veces que se indique


print(repeat_str(3, "a"))
```



Devuelve el String * el numero de veces y luego printear poniendo el valor de cada variable.

10.Convert a Boolean to a String

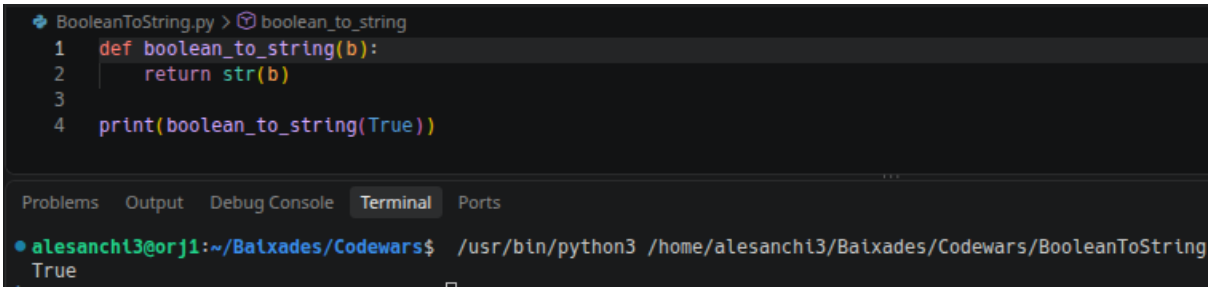
Convertir el valor booleano dado en su representación de cadena.

```python
def boolean_to_string(b):
    """Convierte un valor booleano (True o False) en su
representación en cadena de texto (String).

    PARAMETERS
    ---
    b (bool): El valor booleano que se desea transformar a texto.

    RETURNS
    ---
    str: Devuelve el resultado de aplicar la función `str()` sobre el
booleano, transformándolo en cadena ("True" o "False").
    """
    return str(b)  # método transformación boolean a str (String)


print(boolean_to_string(True))
```



usamos str() y dentro se pone el boolean.

Actividades dificultad 7

1.Leap Years

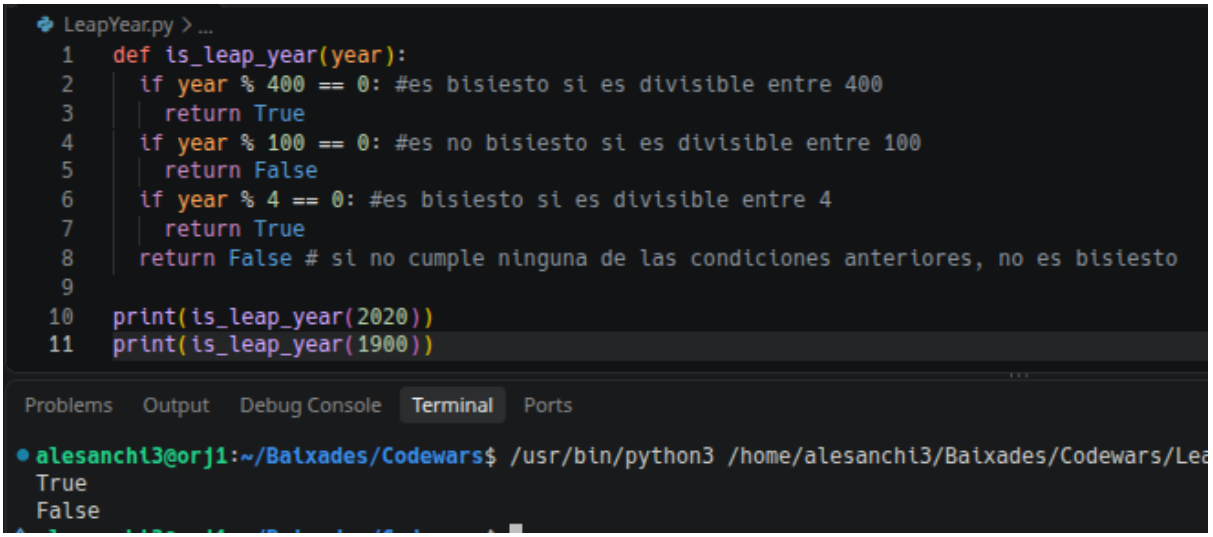
Debe determinar, si un año dado es un año bisiesto o no.

```python
def is_leap_year(year):
    """Determina si un año dado es bisiesto según las reglas del calendario gregoriano.

    PARAMETERS
    ---
    year (int): El año que se desea comprobar.

    RETURNS
    ---
    bool: Devuelve `True` si el año es bisiesto (cumple las condiciones de divisibilidad) o `False` en caso contrario.
    """
    if year % 400 == 0:  # es bisiesto si es divisible entre 400
        return True
    if year % 100 == 0:  # es no bisiesto si es divisible entre 100
        return False
    if year % 4 == 0:  # es bisiesto si es divisible entre 4
        return True
    return False  # si no cumple ninguna de las condiciones anteriores, no es bisiesto


print(is_leap_year(2020))
print(is_leap_year(1900))
```



Si es divisible entre 400 y 4 es bisiesto, si no es divisible entre 400 y entre 100 si o no ha devuelto en ningún if, no es bisiesto.

2.You’re a Square!

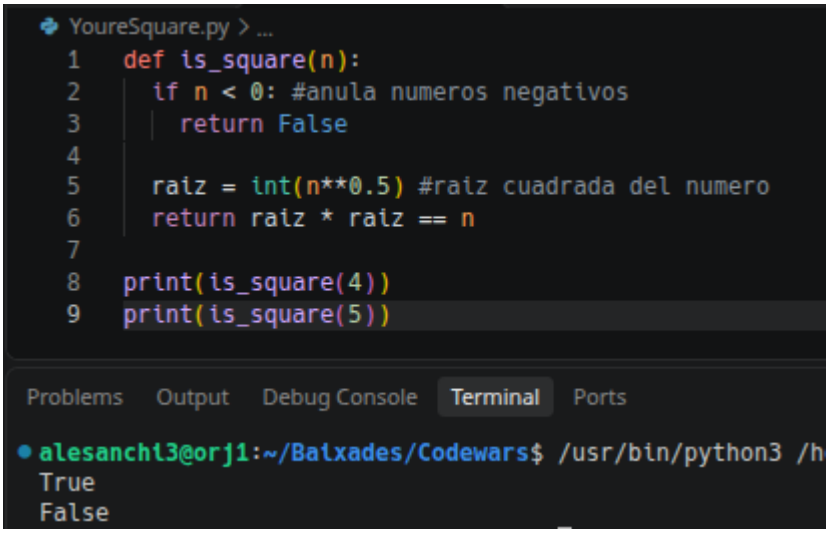
Dado un número entero, determine si es un número cuadrado.

```python
def is_square(n):
    """Verifica si un número es un cuadrado perfecto (es decir, si tiene una raíz cuadrada exacta).

    PARAMETERS
    ---
    n (int/float): El número que se desea comprobar.

    RETURNS
    ---
    bool: Devuelve `False` si el número es negativo, o un valor booleano (`True` o `False`) comprobando si el cuadrado de su raíz entera es igual a `n`.
    """
    if n < 0:  # anula numeros negativos
        return False

    raiz = int(n ** 0.5)  # raiz cuadrada del numero
    return raiz * raiz == n
print(is_square(4))
print(is_square(5))
```



Descarta primero los números negativos y después calcula la raíz cuadrada del número. quita decimales (al ser entero) y multiplicarla por sí mismo, la función si el resultado vuelve a dar el número original es cuadrado.

3.Highest and Lowest

Se le da una cadena de números separados por espacio, y tiene que devolver el número más alto y más bajo.

```
def high_and_low(numbers):
    """Encuentra el número mayor y el menor en una cadena de texto de números separados por espacios.

    PARAMETERS
    ---
    numbers (str): Una cadena de texto que contiene números enteros separados por espacios.

    RETURNS
    ---
    str: Devuelve una cadena de texto con el número mayor y el número menor separados por un guion (ej. "5-1").
    """
    mayor = None
    menor = None
    actual = ""

    for i in numbers + " ":
```

```
        if i != " ":
            actual += i  # hace que actual sea igual a i
        else:
            if actual:
                num = int(actual)
                # guarda a mayor y menor
                if mayor is None or num > mayor:
                    mayor = num
                if menor is None or num < menor:
                    menor = num
                actual = ""

    return str(mayor) + "-" + str(menor)


print(high_and_low("1 2 3 4 5"))
```



Esta función toma una cadena de números separados por espacios, los va leyendo uno por uno y los convierte en enteros. Luego, compara cada número para encontrar cuál es el mayor y cuál es el menor. Finalmente, devuelve ambos números de texto como mayor y menor

4.Descending Order

Función que pueda tomar cualquier entero como un argumento y devolverlo con sus dígitos en orden descendente.

```
def descending_order(num):
    """Reorganiza los dígitos de un número entero en orden
descendente para formar el número más grande posible.

    PARAMETERS
    ---
    num (int): El número entero cuyos dígitos se van a reordenar.

    RETURNS
    ---
    int: Devuelve la variable `resultado` con los dígitos
reorganizados de mayor a menor.
    """
    resultado = 0
    for i in "9876543210":  # guarda los numeros en orden
descendente
        for n in str(num):
            if n == i:
                resultado = resultado * 10 + int(
                    n)  # guarda como un espacio para poder
desplazar los numeros y que no se pierdan
    return resultado


print(descending_order(764832895))
```



Usamos un primer for para recorrer la cadena de texto, dentro usamos otro for para convertir cada dígito de la cadena en un número comparándolos uno a uno con multiplicar por 10 y sumar (resultado = resultado * 10 + int(n)), vamos juntando esos dígitos ordenados de mayor a menor, finalmente devuelve el número

5.Multiply Adjacent Digits

Multiplica los dígitos adyacentes que no están separados por un '-' o un '+'En una cuerda, luego haga la suma.

```
def digit_multiplication(s):
    """Procesa una cadena de texto con operaciones, multiplicando los
dígitos de cada número y aplicando los signos correspondientes.

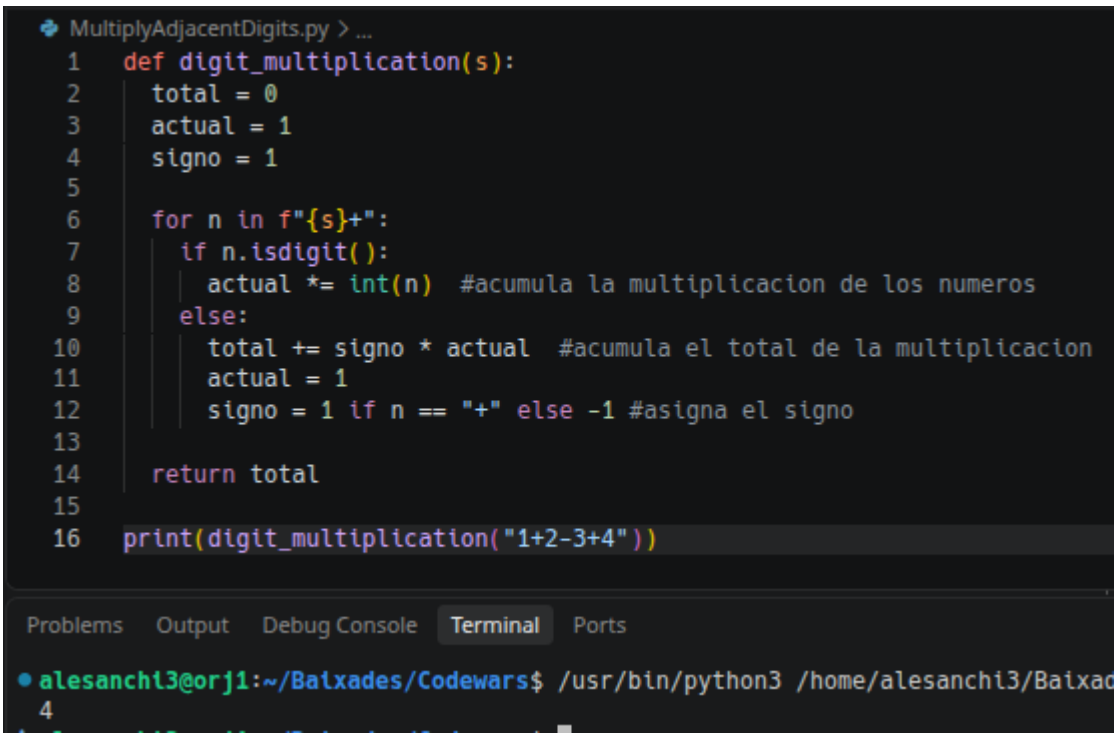
    PARAMETERS
    ---
    s (str): La cadena de texto que contiene la expresión con números y
operadores (`+` o `-`).

    RETURNS
    ---
    int: Devuelve la variable `total` con el resultado numérico final de
la operación acumulada.
    """
    total = 0
    actual = 1
    signo = 1

    for n in f"{s}+":
        if n.isdigit():
            actual *= int(n)  # acumula la multiplicacion de los numeros
        else:
            total += signo * actual  # acumula el total de la
multiplicacion
            actual = 1
            signo = 1 if n == "+" else -1  # asigna el signo

    return total


print(digit_multiplication("1+2-3+4"))
```



Esta función multiplica los dígitos que están juntos y luego suma o resta esos resultados según los signos (+ o -) que los separen. Al final, devuelve el valor total de todas las operaciones combinadas.